

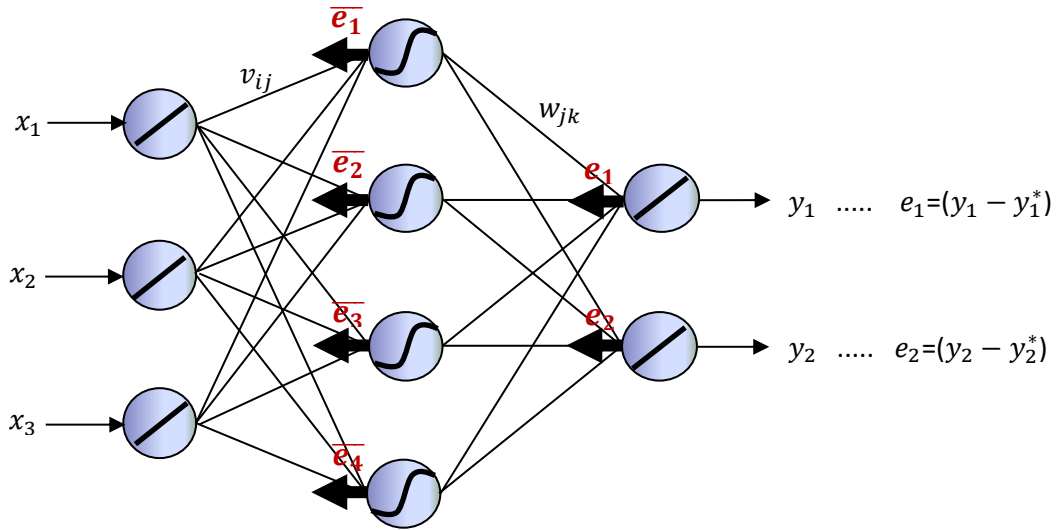
Error Back Propagation Algorithm

Networks coefficients are updated by the steepest descend algorithm with the expression:

$$v_{ij} = v_{ij} - \eta \frac{\partial J}{\partial v_{ij}}$$

$$w_{jk} = w_{jk} - \eta \frac{\partial J}{\partial w_{jk}}$$
(1)

The back-propagation algorithm permits the computation of the derivatives $\partial J / \partial v_{ij}$ and $\partial J / \partial w_{jk}$ using a special arrangement of equations based on back-propagated errors. The error of the network is defined as the difference between network outputs and desired outputs, therefore, the error is known at the output of the output neurons. This error can be back-propagated to the input of the neurons of the output layer and hidden layer, as it is shown in Figure 1: the error at the network output $e_k = y_k - y_k^*$ is back-propagated to the input of the output neurons, and afterwards to the inputs of the neurons of the hidden layer.



According to the back-propagation algorithm the derivatives $\partial J / \partial v_{ij}$ and $\partial J / \partial w_{jk}$ are computed with the following expression:

$$\frac{\partial J}{\partial v_{ij}}, \frac{\partial J}{\partial w_{jk}} = (\text{back propagated error})(\text{output of previous layer neuron})$$
(2)

Since output layer neurons are linear, the back-propagated errors e_1 and e_2 at their inputs are equal to the errors at the output of the neural network. Then the back propagated errors for the output neurons are:

$$e_1 = y_1 - y_1^*$$

$$e_2 = y_2 - y_2^*$$
(3)

These errors may be back-propagated to the input of the neurons in the hidden layer considering the weighting coefficients connecting the corresponding neurons as follows:

$$\bar{e}_1 = (w_{11}e_1 + w_{12}e_2)f'(m_1) \quad \bar{e}_2 = (w_{21}e_1 + w_{22}e_2)f'(m_2)$$

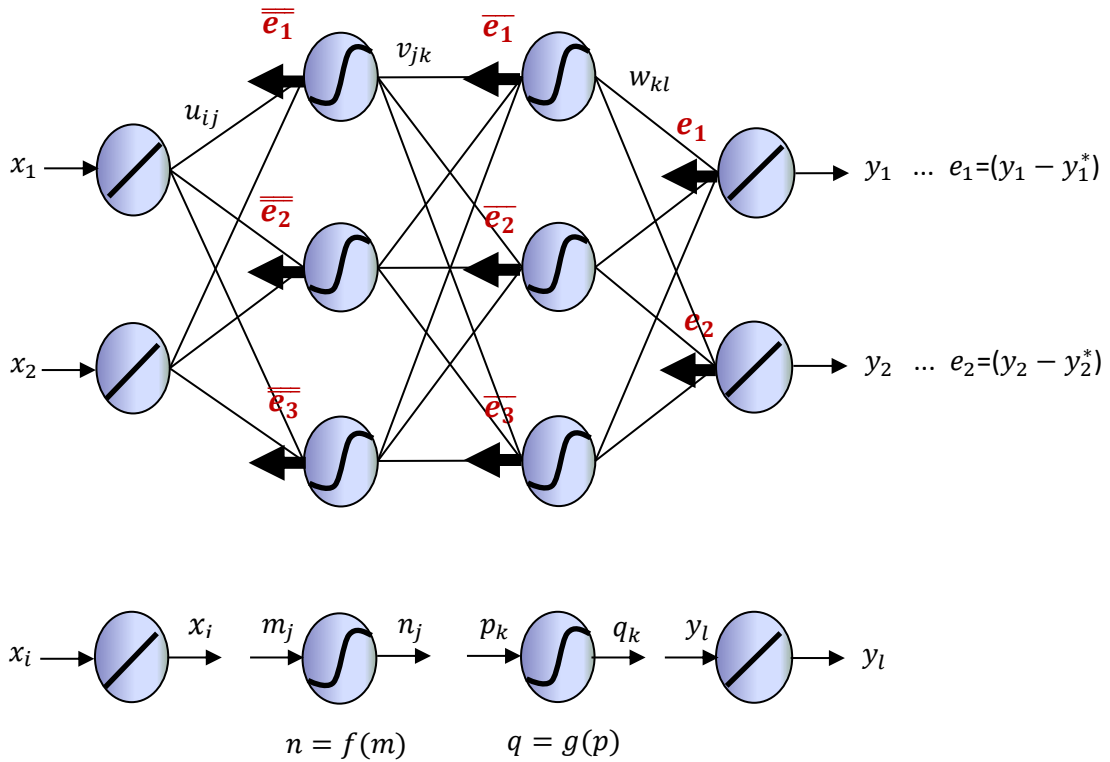
(4)

$$\bar{e}_3 = (w_{31}e_1 + w_{32}e_2)f'(m_3) \quad \bar{e}_4 = (w_{41}e_1 + w_{42}e_2)f'(m_4)$$

Knowing the back-propagated errors e_k and $\bar{e}_j$, the derivatives $\partial J/\partial v_{ij}$ and $\partial J/\partial w_{jk}$ can be calculated from equation (2) as follows:

$$\begin{aligned} \frac{\partial J}{\partial w_{12}} &= e_2 n_1 & \frac{\partial J}{\partial w_{31}} &= e_1 n_3 & \frac{\partial J}{\partial w_{14}} &= e_4 n_1 \\ \frac{\partial J}{\partial v_{12}} &= \bar{e}_2 x_1 & \frac{\partial J}{\partial v_{31}} &= \bar{e}_1 x_3 & \frac{\partial J}{\partial v_{24}} &= \bar{e}_4 x_2 \end{aligned} \quad (5)$$

Proceeding in a similar way for the neural network with two hidden layers shown in Figure 2, the back-propagated errors e_l , $\bar{e}_k$ and $\bar{\bar{e}}_j$ can be computed as in equations (6) to (8).



Errors e_l are computed as follows:

$$\begin{aligned} e_1 &= y_1 - y_1^* \\ e_2 &= y_2 - y_2^* \end{aligned} \quad (6)$$

Errors $\bar{e}_k$ are computed as follows:

$$\bar{e}_1 = (w_{11}e_1 + w_{12}e_2)g'(p_1) \quad \bar{e}_2 = (w_{21}e_1 + w_{22}e_2)g'(p_2) \quad \bar{e}_3 = (w_{31}e_1 + w_{32}e_2)g'(p_3) \quad (7)$$

Errors $\bar{\bar{e}}_j$ are computed as follows:

$$\begin{aligned}
\bar{e}_1 &= (v_{11}\bar{e}_1 + v_{12}\bar{e}_2 + v_{13}\bar{e}_3)f'(m_1) \\
\bar{e}_2 &= (v_{21}\bar{e}_1 + v_{22}\bar{e}_2 + v_{23}\bar{e}_3)f'(m_2) \\
\bar{e}_3 &= (v_{31}\bar{e}_1 + v_{32}\bar{e}_2 + v_{33}\bar{e}_3)f'(m_3)
\end{aligned} \tag{8}$$

With the back propagated errors e_l , $\bar{e}_k$ and $\bar{e}_j$, the derivatives are calculated from the following equations:

$$\begin{aligned}
\frac{\partial J}{\partial w_{12}} &= e_2 q_1 & \frac{\partial J}{\partial w_{31}} &= e_1 q_3 & \frac{\partial J}{\partial w_{22}} &= e_2 q_2 \\
\frac{\partial J}{\partial v_{12}} &= \bar{e}_2 n_1 & \frac{\partial J}{\partial v_{31}} &= \bar{e}_1 n_3 & \frac{\partial J}{\partial v_{24}} &= \bar{e}_4 n_2 \\
\frac{\partial J}{\partial u_{12}} &= \bar{e}_2 x_1 & \frac{\partial J}{\partial u_{21}} &= \bar{e}_1 x_2 & \frac{\partial J}{\partial u_{23}} &= \bar{e}_3 x_2
\end{aligned} \tag{9}$$

If pattern learning is applied, then the derivatives of equations (5) and (9) can be used for updating the weighting coefficients. However, if batch learning is applied, the derivatives are added for each input-output data for taking the average which is used for updating the weighting coefficients.

Problem.

For a neural network with three hidden layers, determine the expressions for computing the back-propagated errors, and the derivatives of the cost function J respect to the weighting coefficients.

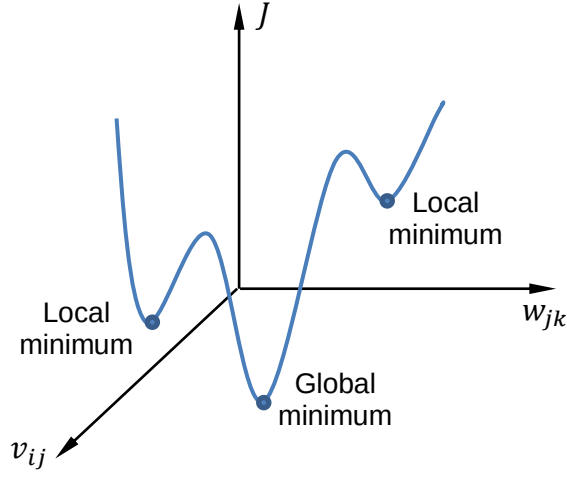
Local and Global Minimum

When training a neural network the objective is to find the weighting coefficients that minimize the cost function J . For a one-hidden-layer neural network, the objective is to find all weighting coefficients v_{ij} and w_{jk} that minimize J . The minimum is found when the following conditions are satisfied:

$$\begin{aligned}
\frac{\partial J}{\partial w_{jk}} &= 0 \\
\frac{\partial J}{\partial v_{ij}} &= 0
\end{aligned} \tag{10}$$

However, it could happen that these conditions are satisfied but the desired minimum of J has not been attained. Figure 3 shows a possible functional dependence of the cost function J respect to the weighting coefficients v_{ij} and w_{jk} . It is noted that there are several points that satisfy equations (10): some points are local minimum, local maximum, or global minimum.

Although the objective of the network training is to find the global minimum, the network could get trapped into a local minimum: the derivatives $\partial J / \partial v_{ij}$ and $\partial J / \partial w_{jk}$ are equal to zero but the training converges to undesired values of weighting coefficients v_{ij} and w_{jk} and with a high value of the cost function J . This situation is common when training neural networks, and it is important to take proper actions to avoid local minimum trapping as possible.



As it was derived in a previous section, the equation for computing the derivative $\partial J / \partial v_{ij}$ for a neural network with one output is given by the following expression:

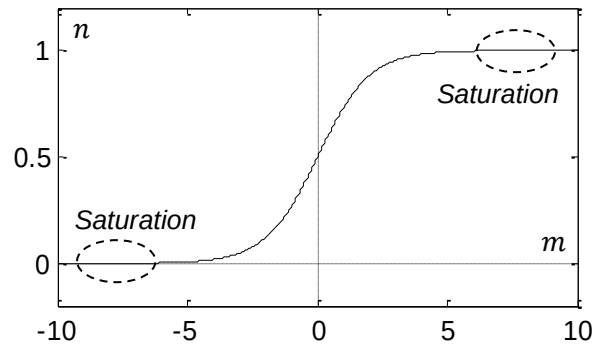
$$\frac{\partial J}{\partial v_{ij}} = (y - y^*) w_{j1} f'(m_j) x_i \quad (11)$$

where y and y^* are the network output and desired output, respectively, w_{j1} is weighting coefficient between the j^{th} hidden neuron and output neuron, $f(m_j)$ is the activation function of the j^{th} neuron of the hidden layer, and x_i is the input to the i^{th} input neuron.

In equation (11) it is noted that for derivative $\partial J / \partial v_{ij}$ be equal to zero it is desired that $y = y^*$. However, it could happen that $\partial J / \partial v_{ij}$ is equal to zero even when $y \neq y^*$ but $f'(m_j) = 0$. If this situation happen, the network training has converged getting trapped into a local minimum with high value of the cost function J and undesired values of weighting coefficients v_{ij} and w_{jk} .

Figure 4 shows the drawing of type-1 sigmoid function given by the following expression:

$$n = \frac{1}{1 + e^{-m}}$$



It is noted that $f'(m)$ equals to zero for high absolute values of m where the neuron saturates: the neuron output n is the same for different values of m . One conclusion of this analysis is that in order to

avoid the possibility of local minimum, the values of inputs m to the hidden neurons should be small, say in the range $[-3 \ 3]$ where the derivative $f'(m)$ is different to zero. For a one-hidden-layer neural network with two inputs neurons and three hidden neurons, the input m_j to each hidden neuron is:

$$\begin{aligned} m_1 &= v_{11}x_1 + v_{21}x_2 \\ m_3 &= v_{13}x_1 + v_{23}x_2 \end{aligned} \quad (12)$$

For making m_j small, it is required that both weighting coefficients v_{ij} and network inputs x_i be small. From this analysis the following recommendations are formulated in order to reduce the possibility that the network training gets trapped into local minimum:

- Initialize the weighting coefficients v_{ij} and w_{jk} with small values.
- Ensure the network inputs x_i are small. If it is not the case, the network inputs must be scaled.

In practice, network input and output training data is scaled to small values in the range $[0 \ 1]$, $[-1 \ 1]$, $[-2 \ 2]$, etc. Also, initial values of v_{ij} and w_{jk} are chosen to be in the range $[0 \ 0.5]$, $[-0.5 \ 0.5]$ or smaller.

Scaling of Input and Output Data

Consider a neural network with two inputs and two outputs with maximum and minimum values as given in Figure 5. It is noted that the maximum and minimum values of inputs and outputs are very different: some are very high and others are very low. Trying to train the neural network directly with this data will not yield good results for the following reasons:

- The values of network inputs are very high so that the neuron activation functions $f(m)$ will saturate, and the training will get trapped into a local minimum with a high value of the cost function J .
- The optimization problem (network training) is not balanced and it could take too much for convergence or not converge at all.

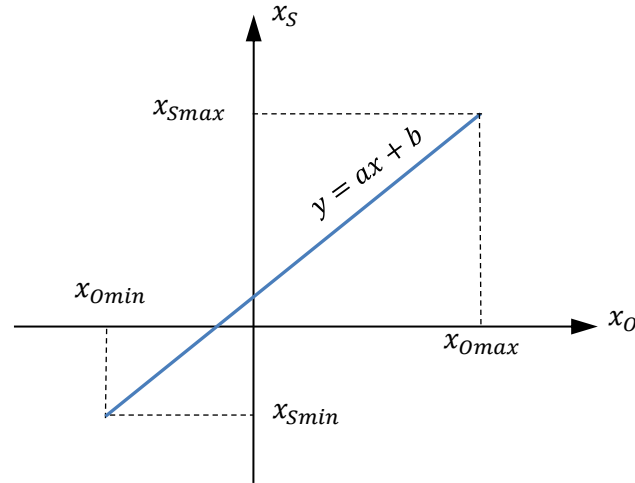
Data					
		x_1	x_2	y_1	y_2
		$x_{1(1)}$	$x_{2(1)}$	$y_{1(1)}$	$y_{2(1)}$
		$x_{1(2)}$	$x_{2(2)}$	$y_{1(2)}$	$y_{2(2)}$
		$x_{1(3)}$	$x_{2(3)}$	$y_{1(3)}$	$y_{2(3)}$
		...	...	...	...
		$x_{1(N)}$	$x_{2(N)}$	$y_{1(N)}$	$y_{2(N)}$
Real bounds	Max	1000	850	0.10	80
	Min	-400	0	0.01	-40
Scaled bounds	Max	2	2	1	1
	Min	-2	-2	-1	-1

For solving the above mentioned problem, input and output data must be scaled to small values of similar ranges so that neuron saturation is avoided and the optimization problem is balanced. The aim of scaling is to transform the input and output data to have new maximum and minimum values. In Figure 5 the desired maximum and minimum bounds of scaled data are shown. Although these values are small, they are only referential. Others small ranges such as $[-1 \ 1]$, $[0 \ 1]$, $[-1 \ 0]$, etc., could also be chosen.

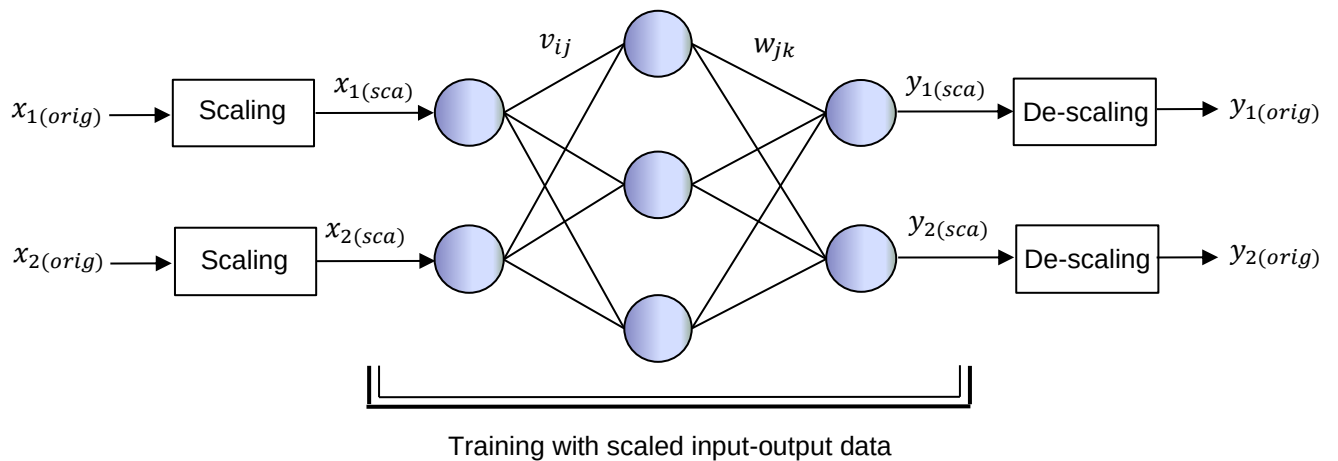
There are several ways to scale data. The simplest way it is to divide each input or output data by the maximum absolute value of its sets of data, respectively. Other way is to perform a linear scaling of the data so that the scaled data is within the desired range.

If x_{Omax} and x_{Omin} are the maximum and minimum values of a given input (or output) data training set, and x_{Smax} and x_{Smin} are the desired maximum and minimum values of the scaled data, a linear transformation can be easily applied as it is shown in Figure 6 where the coefficients of the scaling line $y = ax + b$ can be determine straightforward using the values of x_{Omax} , x_{Omin} , x_{Smax} and x_{Smin} . Suffixes *o* and *s* stand for *original* data and *scaled* data, respectively.

This linear scaling is independently completed for each input and output variable.



It is important to emphasize that the neural network is trained and operates with scaled input and output data as it is shown in Figure 7. Since the output of the neural network is scaled, it has to be de-scaled for recovering the original output data. Linear de-scaling transformations can be formulated in a similar way as linear scaling.



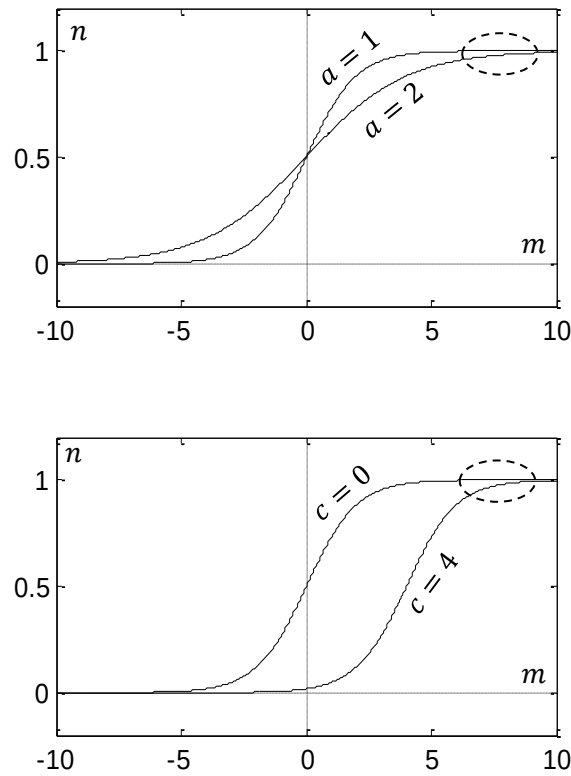
Adaptation of Neuron Activation Functions

As it was pointed out before, one of the reasons why the neural network is trapped into a local minimum is the saturation of the neuron activation function (sigmoid function). Besides the data scaling presented in the previous section, another way to avoid neuron saturation is by modifying the shape of the activation so that saturation is avoided.

For the case of the sigmoid function, one way to avoid saturation is by changing the slope and center of the sigmoid function as it is shown in Figure 8 where a refers to slope and c to center of the sigmoid function. These two additional coefficients can be incorporated in the sigmoid function equation as it shown in the following expression:

$$n = f(m) = \frac{1}{1 + e^{-\left(\frac{m-c}{a}\right)}} \quad (13)$$

From Figure 8 it is noted that for higher values of a , the slope of the sigmoid function is lower so that the values of m generating saturation are higher. Also, for higher values of c , the center of the sigmoid function moves to the right so that the values of m generating saturation are higher. Therefore, by changing the values of the slope a and center c of the sigmoid function, saturation may be avoided.



An interesting point is how to adaptively change the values of slope a and center c . In other words, how to determine the values of a and c by network training in a similar way as the weighting coefficients v and w are updated during the network training process. For that, it is required to compute the derivatives of the cost function J respect to a and c , respectively.

Using the steepest descend algorithm, the slope a and center c of the sigmoid function can be updated from the following expressions where suffixes have been omitted:

$$\begin{aligned} a &= a - \eta \frac{\partial J}{\partial a} \\ c &= c - \eta \frac{\partial J}{\partial c} \end{aligned} \quad (14)$$

For a neural network with one hidden layer and one output neuron y , by directly taking the derivatives of the cost function J respect to slope a and center c , the following equations can be obtained:

$$\frac{\partial J}{\partial a_j} = (y - y^*)w_{j1}n_j(n_j - 1) \frac{(m_j - c_j)}{a_j^2}$$

(15)

$$\frac{\partial J}{\partial c_j} = (y - y^*)w_{j1}n_j(n_j - 1) \frac{1}{a_j}$$

(16)

In general, for a multi-output neural network, the derivatives $\partial J/\partial a$ and $\partial J/\partial c$ can be calculated as follows:

$$\frac{\partial J}{\partial a} = \text{diag} \left[n_j(n_j - 1) \frac{(m_j - c_j)}{a_j^2} \right] W(y - y^*)$$

$$\frac{\partial J}{\partial c} = \text{diag} \left[\frac{n_j(n_j - 1)}{a_j} \right] W(y - y^*)$$

where $\frac{\partial J}{\partial a} = \left[\frac{\partial J}{\partial a_1} \frac{\partial J}{\partial a_2} \frac{\partial J}{\partial a_3} \dots \right]^T$, weight matrix $W = [w_{jk}]$, y and y^* are the network output and desired outputs vectors, respectively, and n_j , m_j , a_j , c_j are values corresponding to the j^{th} neuron of the hidden layer.

Applying Neural Networks to Pattern Recognition Problems

Neural networks can be effectively applied to solve pattern recognition problems: a system is designed to identify to which set a given pattern belongs to. The recognition system is designed using a given group of model patterns belonging to known sets, and afterwards the effectiveness of the recognition is verified using validation patterns different to the model patterns, but belonging to any of the defined sets.

For instance, the sets could 10 people, the model patterns could be 5 different faces of each of the 10 people, and the validation patterns could be different faces of the 10 people. The recognition system should be able to identify to which of the 10 people the validation face belongs to.

